Assignment 2: Paper Summarization and Reproduction

HyperImpute: Generalized Iterative Imputation with Automatic Model Selection
Jarrett, Cebere, Liu, Curth, van der Schaar (ICML 2022)

Yann Klees 0800619
Data Wrangling, Utrecht University

24 October 2026

Part 1. Paper summary

1. Main problem

Existing imputation methods fall into two families, and each has a strength the other lacks. Iterative imputers such as MICE [3] and MissForest [4] fit a flexible model to every column. They also work on incomplete data when values are missing at random (MAR), so that missingness depends only on observed values. The catch is that each column model has to be chosen and tuned by hand. Deep generative imputers such as GAIN [5] and MIWAE [6] learn one joint model, so no model has to be specified per column. They are hard to train, however, and most need complete training data or assume that values are missing completely at random (MCAR). HyperImpute [1] aims to combine flexible models per column, automatic configuration and training on incomplete data under MAR.

2. Proposed solution

HyperImpute keeps the column-by-column structure of iterative imputation. The difference is that the user no longer fixes one model in advance. For each column, the method searches over several model classes and their hyperparameters (Section 3.2). Algorithm 1 (Section 3.4) has four steps.

1. Fill every missing cell with a simple first guess, by default the column mean.
2. Visit the columns one at a time. Train a model on the rows where the column is observed, with all other columns, including their current guesses, as inputs.
3. Try several learners and hyperparameters, keep the one that predicts the column best, and use it to fill the column's missing rows.
4. Repeat over all columns until the imputations stop changing.

The candidate pool holds linear and logistic regression, random forests, XGBoost [7], CatBoost [8] and small neural networks. Continuous columns get regressors and categorical columns get classifiers (Section 4, Appendices A.3 and A.6). If the search is switched off and each column gets a fixed model, the algorithm becomes classic iterative imputation. HyperImpute therefore generalises it.

3. Optimization techniques and special cases

Section 3.3 argues that the key design choice is where the model search sits relative to the imputation loop. A top-down search reruns the whole imputation for every combination of column models. With D columns and A configurations each, that means A^D runs, which is far too expensive. A concurrent search tunes each column on its own at a cost of $O(KDA)$ for K rounds. It ignores, however, how one column's model affects the others. HyperImpute turns this around. The imputation loop runs on the outside, and a small search for each column runs inside it. The cost stays at $O(KDA)$, yet every search sees the latest imputations of all other columns.

By default, the search is an adapted Hyperband [9] (Section 4, Appendix A.4). Hyperband trains many candidates briefly, drops the weak ones and gives the survivors more training. Neural networks count training in epochs and forests in trees, so both are mapped onto one common budget. Naive search and Bayesian optimisation are also available. Later rounds can skip the search and reuse earlier choices. The loop stops after a maximum number of rounds, when the imputations barely change, or when the objective stops improving (Appendix A.5).

4. Strengths and limitations

Strengths.

- It removes the manual model choice that holds iterative imputation back, and it inherits the properties of iterative imputation under MAR.
- Unlike most deep generative imputers, it trains on incomplete data and handles mixed data types (Table 1).
- An ablation (Table 2) credits a performance gain to each design component.

Limitations.

- Nothing guarantees that the separate column models form a valid joint distribution over all variables.
- It returns a single imputation, so it does not quantify how uncertain the filled values are.
- MNAR lies outside the paper's scope, and the paper gives no theory for it.
- Most ablation gaps between HyperImpute and its stronger variants are smaller than one standard deviation, and the paper reports no significance tests or runtimes. Table 7 even marks HyperImpute best on white wine (WD 0.426), although column_naive (0.390) and global_search (0.410) are lower.

5. Evaluation methodology

The authors remove values artificially from 12 UCI datasets [10], so the true values are known. They simulate MCAR, MAR and MNAR at rates from 10% to 70%, with MAR at 30% as the main setting. HyperImpute is compared with eight baselines, among them MICE, MissForest, GAIN and MIWAE. Errors are measured with RMSE and the Wasserstein distance (WD), averaged over 10 seeds.

HyperImpute is best on both metrics on 10 of the 12 datasets and wins at least one metric on every dataset. In the ablation, it lowers RMSE by 18% on average compared with the best single-learner variant. The search picks simple models on small datasets and at high missingness, and boosting and neural networks on large ones. The imputations settle after about four rounds.

References

[1] D. Jarrett et al. HyperImpute: Generalized iterative imputation with automatic model selection. *ICML*, 2022.

[2] HyperImpute source code. `https://github.com/vanderschaarlab/hyperimpute`

[3] S. van Buuren, K. Groothuis-Oudshoorn. mice: Multivariate imputation by chained equations in R. *J. Stat. Softw.* 45(3), 2011.

[4] D. J. Stekhoven, P. Bühlmann. MissForest: Nonparametric missing value imputation for mixed-type data. *Bioinformatics* 28(1), 2012.

[5] J. Yoon, J. Jordon, M. van der Schaar. GAIN: Missing data imputation using generative adversarial nets. *ICML*, 2018.

[6] P.-A. Mattei, J. Frellsen. MIWAE: Deep generative modelling and imputation of incomplete data sets. *ICML*, 2019.

[7] T. Chen, C. Guestrin. XGBoost: A scalable tree boosting system. *KDD*, 2016.

[8] L. Prokhorenkova et al. CatBoost: Unbiased boosting with categorical features. *NeurIPS*, 2018.

[9] L. Li et al. Hyperband: A novel bandit-based approach to hyperparameter optimization. *JMLR* 18, 2018.

[10] D. Dua, C. Graff. UCI Machine Learning Repository, 2019.

[11] F. Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin* 1(6), 1945.

Part 2. Reproduction

Code: `https://github.com/YannKleees/hyperreimpute`.
I use the authors' `hyperimpute` package [2], version 0.1.17, with the settings of their notebook `experiments_04_gain_of_function.ipynb`. As a check, I also use version 0.1.5 (July 2022), which keeps the CatBoost, random forest and XGBoost defaults of the first public release (0.1.1, May 2022). All runs were on CPU.

```
git clone https://github.com/YannKleees/hyperreimpute.git && cd hyperreimpute
pip install -r requirements.txt
python -m hyperreimpute.experiments    # writes results/*.csv
python -m hyperreimpute.version_check  # here and in venv_015, see README
```

1. Selected experiment

I reproduce the source of gains ablation (Table 2 of [1]). It tests the central claim that choosing a model for each column improves iterative imputation. As in the paper, 30% of the values are removed under MAR, with 10 seeds. I use airfoil, compression and white wine, since california and letter were too slow on a laptop. Two extra controls use a CatBoost with 1000 trees and learning rate 0.03, close to CatBoost's own defaults. On the same masks, I also run six of the paper's eight benchmarks to check its headline result (Figure 2).

2. Comparison with the paper

Table 1. Imputation error

	airfoil		compression		white wine	
Method	Paper	Ours	Paper	Ours	Paper	Ours
ice_lr	2.35	1.95	1.43	1.49	0.92	0.86
ice_rf	2.37	1.91	1.62	1.68	0.99	0.90
ice_cb	2.08	2.55	1.25	2.29	0.88	1.10
global_search	1.60	1.92	1.03	1.45	0.85	0.79
column_naive	1.86	1.32	1.09	1.14	0.85	0.79
wo_flexibility_cb	1.59	2.16	1.08	2.00	0.88	1.10
wo_adaptivity	1.66	1.92	1.05	1.58	0.89	0.79
HyperImpute	1.48	1.80	1.01	1.48	0.80	0.79
ice_cb_fixed	n/a	**1.25**	n/a	**1.04**	n/a	**0.75**
column_naive_cb_fixed	n/a	**1.25**	n/a	**1.04**	n/a	**0.75**

Row names follow the paper, and wo_flexibility_rf is left out. Results vary a lot between seeds, so I call a difference clear only if it holds on almost every seed, judged with paired Wilcoxon signed-rank tests [11] on the same masks.

The headline result holds. HyperImpute beats mean imputation, ICE, MissForest, SoftImpute, GAIN and MIWAE on all three datasets though the gap is only clear on white wine and compression.

The simple baselines reproduce only in part. On airfoil they score better than in the paper, which suggests the masks differ. On the other two datasets they match the paper closely.

Choosing a model per column clearly beats a single linear model or random forest. It does not beat a well-configured CatBoost, and when that CatBoost joins the pool, selection simply picks it. Selection therefore helps only as much as the other learners are weak. The paper's large gain over its own single-learner baselines shrinks to a few percent here, and only white wine matches.

Full HyperImpute reproduces only on white wine. On the other two datasets its error is clearly higher than reported. Tuning even makes it worse than per-column selection without tuning, and it makes each run several times slower. The paper reports the opposite.

3. Parameters

All runs use hyperimpute 0.1.17, or 0.1.5 for the check, with pandas below 2.3 and Python 3.11 on CPU. Data are min-max scaled and masked under MAR at 30% with column_limit=10. The seeds are 0 to 9, or 0 to 4 in the check, and each imputer gets random_state equal to the seed, where

the notebook keeps 0. RMSE uses the masked cells, and the Wasserstein distance is summed over columns.

Table 2. Settings.

Method	Optimizer	Column	Iteration	Lazy	Learners
HyperImpute	hyperband	yes	yes	no	full pool
global_search	hyperband	no	no	yes	full pool
wo_adaptivity	hyperband	yes	no	yes	full pool
wo_flexibility_cb	hyperband	no	no	yes	CatBoost only
column_naive	simple	yes	no	yes	full pool
ICE variants	simple	no	no	yes	one fixed learner
ice_cb_fixed	simple	no	no	yes	CatBoost, 1000 trees, learning rate 0.03
column_naive_cb_fixed	simple	yes	no	yes	full pool with this CatBoost

The full pool holds linear models, random forest, XGBoost and CatBoost; hyperimpute adds neural networks only when CUDA is available, so my CPU runs had none. column_naive is not in the authors' notebook, so I built it from the paper's Table 3. The notebook's wo_flexibility selects one global model once, although the paper's Table 2 marks it column-wise and adaptive; I follow the notebook. Imputation starts from the column mean and visits columns in random order, for at most `n_inner_iter=40` rounds, with `select_patience=5` and `optimize_thresh=5000`. Other settings keep the defaults.

4. Reasons for the differences

For me the most likely cause is that the hyperimpute package changed after the paper came out. The current version gives CatBoost only 10 trees and a very small learning rate by default. Its predictions therefore barely move away from the column mean: ice_cb is within 0.4% of mean imputation on all three datasets. This explains the weak CatBoost baseline and the CatBoost-only variant. Version 0.1.5 left the learning rate to CatBoost itself, and the paper's list of CatBoost settings (Table 5) has none either. Later versions also changed the search ranges of CatBoost and XGBoost, the default size of the random forest and the number of iterations. The missing neural networks and the global wo_flexibility are smaller deviations.

To test this, I reran HyperImpute, per-column selection and the CatBoost baseline with version 0.1.5 on airfoil and compression, with five seeds and the same masks. That environment also has older CatBoost, XGBoost and scikit-learn, so the check compares environments. The CatBoost baseline improves strongly and gets close to the paper on compression. HyperImpute improves on every seed, by about a fifth, and its Wasserstein distance now matches the paper. It still trails per-column selection slightly, and that method barely changes. The older environment therefore explains the CatBoost results and the gap in Wasserstein distance. It does not explain why tuning fails.

5. Conclusion

HyperImpute beats the standard benchmarks, and per-column selection beats a single linear model or random forest, but not a well-configured CatBoost. Reproductions should fix the package version and check every learner's settings.